Assignment

Topics Covered: Why Pydantic is Required | Data Validation | Field Types

Question 1: What is Pydantic in Python? Explain its main purpose and why it is useful when working with structured data.

Answer:

Pydantic is a Python library used for data validation and data parsing. It ensures that the data follows as predefined structure and contains the correct data types.

Main Purpose of Pydantic :

The main purpose of Pydantic is to validate structured data, check data types, and convert data into the required format. It helps developers ensure that the input data is correct and reliable.

Why is Pydantic Useful?

1. Data Validation : Checks whether the input data follows the defined rules.
2. Type Checking : Ensures that variables contain the expected data types.
3. Error Handling : Provides clear error messages when invalid data is entered.
4. Structured Data : Helps manage data in a well-organized format using Python classes.

Example :

```
From pydantic import BaseModel

Class Student(BaseModel):
        name : str
        age : int
        marks : float

student = Student(
        name=”Rahul”,
        age=21,
        marks=85.5
)
print (student)
```

Output : name=’Rahul’ age=21 marks=85.5

Question 2: Why is Pydantic required in modern Python applications? Explain at least four benefits of using Pydantic models.

Answer:

Pydantic is required in modern Python applications because it help validate, parse, and manage structured data easily. It ensures that the data received by an application is correct and follows the expected format.
Pydantic uses Python type hints to define data structures and automatically validates the systems.

Benefits of using Pydantic models :

1. **Data Validation :** Pydantic automatically checks whether the input data ,atches the defined data types and rules.
2. **Type Safety :** It ensure that variables contain the expected data types, such as integers, strings, lists.
3. **Automatic Type Conversion :** Pydantic can convert compatilib input values into the required data types automatically.
4. **Error Handling :** It provides clear and detailed error messages when invalid data is provided.
5. **Easy Data Management :** Pydantic models make it easier to organize, store, and manage structured data.
6. **JSON Support :** Pydantic allows easy conversion of python object into JSON format and vice versa.

Example:

```
From pydantic import BaseModel

class Student(BaseModel):
        name: str
        age: int
        marks: float

student = student(
        name=”Vansh”,
        age=22,
        marks=85.5
)
print (student)
```

Output : name=’Vansh’ age=22 marks=85.5

Question 3: What is data validation? Explain how Pydantic validates input data and handles invalid values.

Answer:

Data Validation is the process of checking whether the input data is correct, complete, and follows the required format and rules. It helps prevent incorrect or invalid data from entering an application.

How Pydantic validates input data:

Pydantic uses Python type hints and models to validate inout ata automatically. We define the expected data types in a model, and Pydantic checks the input values when an object an object is created.

If the input data is valid Pydantic creates the object successfully. If the input data is invalid, it raise a ValidationError.

How Pydantic handles invalid values :

1. **Type Checking :** It checks whether input values match the defined data types.
2. **Type Conversion :** It converts compatible input values into the required data types.
3. **Error Handling :** It raises a ValidationError when input data does not meet the validation rules.
4. **Detailed Error Messages :** It provides information about which field contains invalid data and why it is invalid.

Question 4: Create a Pydantic model named User with fields for name, age, email, and is_active. Explain the expected field types.

Answer:

A Pydantic model named user can be created using the BaseModel class. It defines the structure of user data and validations the input according to the specified field types.

Python Program:

```
From pydantic import BaseModel, EmailStr

Class User(BaseModel):
        name: str
        age: int
        email: EmailStr
        is_active: bool
```

Expected Field Types :

1. **name (str) :** Stores the user's name as a string.

2. **age (int) :** Stores the user's age as an integer.
3. **email (EmailStr) :** Stores and validates the user's email address.
4. **is_active (bool) :** Stores the user's active status as True or False.

Question 5: Explain the difference between Python type hints and Pydantic data validation. Give a suitable example.

Answer:

Python Type Hints Pydantic Data Validation are used to define and manage data types in Python, but they have different purposes.

Difference between Python Type Hints and Pydantic Data Validation

Python Type Hints	Pydantic Data Validation
Used to indicate the expected data type of a variable.	Used to variable input data at runtime.
Does not automatically check data types at runtime.	Automatically checks input data at runtime.
Mainly helps developers understand code.	Helps ensure data follows defined rules.
Does not generate validation errors by itself.	Raises validation errors for invalid data.
Used through Python annotations.	Uses models and type hints for validation.

Example :

1. **Python type Hints :**

```
age : int = "twenty"
print (age)
```

Python allows this assignment because type hints do not enforce runtime validation.

2. **Pydantic Data Validation :**

```
from pydantic import BaseModel

class User(BaseModel) :
        age: int
```

user = User(age=”twenty”)
print (user)

Output : Pydantic raise a ValidationError because “twenty” cannot be converted into an integer.

Question 6: What are Field Types in Pydantic? Explain commonly used types such as str, int, float, bool, list, and Optional fields.

Answer:

Field Types in Pydantic define the type of data that can be stored in each field of a Pydantic model. Pydantic uses Python type hints to validate the input data and ensure that it follows the expected data types.

Commonly Used Field Types in Pydantic :

1. **str (string) :** USed to store text values such as names, addresses, and usernames.
2. **int (Integer) :** Used to store whole numbers such as age, marks , and quantity.
3. **float :** Used to store decimal numbers such as price, percentage, and salary.
4. **bool (Boolean) :** Used to store multiple values in a single field. It can contain strings, integers, or other data types.
5. **Optional Fields :** Used when a field can contain a value or None. It is useful when some information is not compulsory.

Question 7: What happens when required data is missing from a Pydantic model? Explain with a suitable example and validation error scenario.

Answer:

When required data is missing from a Pydantic model, Pydantic raises a ValidationError. This happens because all required fields must have values when creating a model object.

What happens when required data is missing?

1. Pydantic checks whether all required fields are provided.
2. If a required field is missing, it does not create the model object.
3. It raise a ValidationError.
4. The error message identifies which required field is missing.

Example :

From import BaseModel, ValidationError

```
Class User (BaseModel):
        name: str
        age: int
        email: str

try:
        user = User(
                name=”Vansh”,
                age=22
        )
expect ValidationError as e:
        print(e)
```

Output : 1 validation error for User email
Field required

Question 8: Explain how default values work in Pydantic models. Create an example where a field has a default value.

Answer:

Default values in Pydantic models are used to assign predefined values to field when the user does not provide any value for them.

If a field has a default value, Pydantic automatically uses that value when creating the model object if the field is missing.

How Default Values Works:

1. A default value is assigned to a field during model definition.
2. If the user provides a value, Pydantic uses the provided value.
3. If the user does not provide a value, Pydantic uses the default value.
4. Field without default values are considered required.

Example :

```
From pydantic import BaseModel

Class User (BaseModel):
        name: str
        age: int = 18
        is active : bool = True
```

user = User (name=”Vansh”)
print (user)

Output : name=’Vansh’ age=18 is_active=True

Question 9: Write a Pydantic model for a Student containing name, age, marks, and skills. Explain how each field is validated.

Answer:

A Pydantic model named Student can be created using the BaseModel class. It contains four field : name, age, marks, and skills. Pydantic automatically validates each according to its defined data type.

Program :

```
From pydantic import BaseModel

class Student (BaseModel):
    name: str
    age: int
    marks: float
    skills: list [str]
```

How Each Field is validated:

1. **name (str) :** Validates that the student’s name is a string.
2. **age (int) :** Validates that the student’s age is a whole number.
3. **marks (float) :** Validates that the student’s marks are a decimal or numeric value.
4. **skills (list[str]) :** Validates that skills are provided as a list containing string values.

Question 10: Explain how Pydantic models can make API or application data safer and easier to manage. Include a simple Python example.

Answer:

Pydantic models make API and application data safer and easier to manage by validating input data, checking data types, and ensuring that the data follows the required structure.

Benefits of Pydantic Models:

1. **Data Validation :** Checks whether the input data is correct and follows the defined rules.
2. **Type Safety :** Ensure that fields contain the expected data types.

3. **Error Handling :** Provides clear error messages when invalid data is entered.
4. **Data Management :** Organizes application data into structured models.
5. **API Security and Reliablity :** Helps prevent unexpected or incorrectly formatted data from being processed by an application.

Example :

```
from pydantic import BaseModel, EmailStr

class User (BaseModel) :
        name: str
        age: int
        email: EmailStr

User = User (
        name=”Vansh”,
        age=22,
        email=”Vansh@gmail.com”
)
print(user)
```

Output : name=’Vansh’ age=22 email=’vansh@gmail.com’